

Introduction à JavaScript

Objectifs du cours

- comprendre le rôle de JavaScript par rapport à HTML et CSS ;
- déclarer et utiliser des variables ;
- afficher des informations dans la console pour observer/déboguer son code ;
- sélectionner un élément du DOM ;
- réagir à un événement utilisateur (clic) ;
- modifier le texte ou la classe d'un élément ;
- écrire une condition simple.

Séance	Compétences principales
S — JavaScript	C08 – Coder C06 – Valider un système informatique

Partie 1 — Rôle de JavaScript

- **HTML** structure le contenu.
- **CSS** met en forme ce contenu.
- **JavaScript** rend la page **interactive** : il peut réagir à une action de l'utilisateur (clic, saisie, chargement de la page...) et modifier la page **après** qu'elle ait été affichée.

Sans JavaScript, une page HTML/CSS reste figée une fois affichée : rien ne change tant que l'utilisateur ne recharge pas la page.

1.1. Où écrire du JavaScript

Comme pour le CSS, on peut écrire le JavaScript dans la page, ou dans un fichier séparé (recommandé, exactement comme `style.css` pour le CSS).

Fichier séparé (méthode utilisée dans le projet boutique) :

```
<body>
  ...
  <script src="script.js"></script>
</body>
```

Remarque : la balise `<script>` est placée juste avant `</body>`, et non dans `<head>`. Ainsi, le navigateur a déjà lu tout le HTML de la page avant d'exécuter le script — le script peut donc trouver les éléments qu'il doit manipuler.

1.2. Premier script : la console

Avant de modifier une page, il est indispensable de savoir **observer** ce que fait son code. Le navigateur fournit une console (ouverte avec la touche **F12**, onglet *Console*).

```
console.log("Bonjour depuis JavaScript !");
```

Que se passe-t-il si vous ouvrez la console (F12) après avoir chargé une page contenant cette ligne ?

▼ Réponse

Le message s'affiche dans la console, mais **jamais** sur la page elle-même. `console.log()` sert uniquement au développeur pour vérifier que son code s'exécute et voir la valeur de ses variables.

`console.log()` sera votre meilleur outil pour comprendre pourquoi un script « ne marche pas ».

Partie 2 — Les variables

Une variable permet de stocker une valeur pour la réutiliser plus tard.

```
const titre = "Ma boutique";  
let quantite = 1;  
  
console.log(titre);  
console.log(quantite);
```

- **const** : la valeur ne pourra **plus jamais changer** ensuite. À utiliser par défaut.
- **let** : la valeur **pourra changer** au cours du script. À utiliser uniquement quand c'est nécessaire.

Que se passe-t-il si vous essayez de réaffecter une variable déclarée avec **const** ?

```
const prix = 100;  
prix = 150; // ✖ Erreur
```

▼ Réponse

Le navigateur affiche une erreur dans la console (**Assignment to constant variable**). C'est volontaire : **const** protège une valeur qui ne doit pas changer.

Quelques types de valeurs courants :

Type	Exemple
Chaîne de caractères	"Bonjour"
Nombre	42
Booléen	true / false

Partie 3 — Sélectionner un élément de la page (le DOM)

Le **DOM** (*Document Object Model*) est la représentation de la page HTML que JavaScript peut lire et modifier. Pour agir sur un élément, il faut d'abord le **sélectionner**.

```
<button id="themeButton" class="theme-toggle">☾ Mode sombre</button>
```

```
const bouton = document.querySelector(".theme-toggle");  
console.log(bouton);
```

`querySelector()` accepte n'importe quel sélecteur CSS déjà connu :

Sélecteur CSS	Exemple
Classe	<code>document.querySelector(".theme-toggle")</code>
Identifiant	<code>document.querySelector("#themeButton")</code>
Balise	<code>document.querySelector("button")</code>

Remarque : Il existe aussi `document.getElementById("themeButton")`, réservé aux identifiants. Les deux fonctionnent ; `script.js` du projet boutique utilise `querySelector`, car c'est la méthode la plus polyvalente (elle fonctionne avec n'importe quel sélecteur CSS).

Que se passe-t-il si le sélecteur ne correspond à aucun élément de la page ?

▼ Réponse

`querySelector()` renvoie `null`, et non une erreur immédiate. L'erreur n'apparaît que si l'on essaie ensuite d'utiliser ce `null` comme un élément (par exemple `null.addEventListener(...)`) — d'où l'intérêt de vérifier avec `console.log()` que l'élément a bien été trouvé.

Partie 4 — Réagir à un événement

Un **événement** est une action qui se produit sur la page : un clic, une saisie clavier, un chargement de page... JavaScript permet d'exécuter du code **quand** un événement se produit.

```
const bouton = document.querySelector(".theme-toggle");

bouton.addEventListener("click", function () {
  console.log("Le bouton a été cliqué !");
});
```

- `"click"` : le type d'événement écouté (il en existe beaucoup d'autres : `"keydown"`, `"submit"`, `"mouseover"`...).
- `function () { ... }` : le code à exécuter à chaque fois que l'événement se produit.

Remarque : On trouve aussi très souvent cette même écriture sous une forme plus courte, la **fonction fléchée** :

```
bouton.addEventListener("click", () => {
  console.log("Le bouton a été cliqué !");
});
```

Les deux écritures sont équivalentes ici. `script.js` du projet boutique utilise cette seconde forme.

Partie 5 — Modifier la page

5.1. Modifier un texte

```
const paragraphe = document.querySelector("p");
paragraphe.textContent = "Le texte a été modifié par JavaScript.";
```

5.2. Modifier une classe CSS

Plutôt que de modifier des styles un par un en JavaScript, on préfère **ajouter/retirer une classe CSS** déjà définie dans `style.css` (le CSS reste responsable de l'apparence, JavaScript ne fait que déclencher le changement) :

```
element.classList.add("ma-classe"); // ajoute la classe
element.classList.remove("ma-classe"); // retire la classe
element.classList.toggle("ma-classe"); // ajoute si absente, retire si présente
element.classList.contains("ma-classe"); // true / false
```

Partie 6 — Les conditions

Une condition permet d'exécuter du code **seulement si** une situation est vraie.

```
const nombre = 7;

if (nombre > 5) {
  console.log("Le nombre est supérieur à 5");
} else {
  console.log("Le nombre est inférieur ou égal à 5");
}
```

Pour comparer deux valeurs, on utilise `===` (égal à) et `!==` (différent de) plutôt que `=` (qui sert à *affecter* une valeur, pas à comparer) :

```
if (document.body.classList.contains("dark-mode")) {
  console.log("Le mode sombre est actif");
}
```

TP — Compteur de clics

Consigne

Créer une page contenant :

- un titre ;
- un paragraphe affichant `Vous avez cliqué 0 fois.` ;
- un bouton `Cliquez ici.`

Chaque clic sur le bouton doit :

1. incrémenter un compteur ;
2. mettre à jour le texte du paragraphe (`Vous avez cliqué X fois.`) ;
3. si le compteur atteint 5, ajouter une classe CSS `.succes` qui change la couleur du paragraphe (à vous de la définir dans `style.css`).

Aide

```
let compteur = 0;

const bouton = document.querySelector("#monBouton");
const paragraphe = document.querySelector("#monParagraphe");

bouton.addEventListener("click", () => {
  compteur = compteur + 1;
  paragraphe.textContent = "Vous avez cliqué " + compteur + " fois.";

  if (compteur >= 5) {
```

```
    paragraphe.classList.add("succes");  
  }  
});
```

Question :

Pourquoi utilise-t-on **let** pour **compteur** et non **const** ?

▼ Réponse

Parce que sa valeur **change** à chaque clic ; **const** interdirait cette réaffectation.

Récapitulatif

Notion	Exemple
Variable	<code>const, let</code>
Déboguer	<code>console.log()</code>
Sélectionner un élément	<code>document.querySelector()</code>
Écouter un événement	<code>addEventListener("click", ...)</code>
Modifier un texte	<code>textContent</code>
Modifier une classe	<code>classList.add/remove/toggle/contains()</code>
Condition	<code>if / else, ===</code>
